

Part B : Data Acquisition

Source 1: Generate dataset: -

```
dt.head()
```

	customer_id	age	gender	region	education_level	employment_type	annual_income	loan_amount	loan_purpose	credit_score	repayment_history	transaction_count
0	1	59	Female	East	High School	Unemployed	213261	526963	Business	815	8	
1	2	49	Male	West	Post-Graduate	Salaried	1325285	765732	Business	696	0	1
2	3	35	Male	East	Graduate	Salaried	568760	861527	Education	405	4	
3	4	28	Female	West	Graduate	Salaried	1053605	361385	Car	384	2	3
4	5	41	Male	South	High School	Salaried	475796	677407	Car	513	5	3

Load dataset: -

```
dt.head()
```

	customer_id	age	gender	region	education_level	employment_type	annual_income	loan_amount	loan_purpose	credit_score	repayment_history	transaction_count
0	1	59	Female	East	High School	Unemployed	213261	526963	Business	815	8	
1	2	49	Male	West	Post-Graduate	Salaried	1325285	765732	Business	696	0	1
2	3	35	Male	East	Graduate	Salaried	568760	861527	Education	405	4	
3	4	28	Female	West	Graduate	Salaried	1053605	361385	Car	384	2	3
4	5	41	Male	South	High School	Salaried	475796	677407	Car	513	5	3

Source 2: JSON File: -

Import JSON: -

```
import json

data = [
    {"customer_id": 101, "gender": "Male", "region": "North"},
    {"customer_id": 102, "gender": "Female", "region": "South"}
]

with open("customer_metadata.json", "w") as f:
    json.dump(data, f, indent=4)

print("JSON file created successfully")
```

JSON file created successfully

Read JSON: -

```

json_df.head()
]:
  customer_id  gender region
0           1    Male  North
1           2  Female  South
2           3    Male   East
3           4  Female   West
4           5    Male  North

```

Source 3 : SQL Database: -

Create database:

Insert data:

```

# Create Database

# insert data
cursor.execute("""
INSERT INTO repayment_history
VALUES(101,12)
""")

cursor.execute("""
INSERT INTO repayment_history
VALUES(102,8)
""")

cursor.execute("""
INSERT INTO repayment_history
VALUES(103,10)
""")

conn.commit()

# read sql table
sql_df = pd.read_sql(
"SELECT * FROM repayment_history",
conn
)

sql_df.head()

```

```

  customer_id  payments
0           101         12
1           102          8
2           103         10
3           101         12
4           102          8

```

Source 4 : Dummy API

Example API:

```
0]:
```

	id	name	username	email	address	phone	website	company
0	1	Leanne Graham	Bret	Sincere@april.biz	{street: 'Kulas Light', 'suite': 'Apt. 556', ...	1-770-736-8031 x56442	hildegard.org	{name: 'Romaguera-Crona', 'catchPhrase': 'Mu...
1	2	Ervin Howell	Antonette	Shanna@melissa.tv	{street: 'Victor Plains', 'suite': 'Suite 87', ...	010-692-6593 x09125	anastasia.net	{name: 'Deckow-Crist', 'catchPhrase': 'Proac...
2	3	Clementine Bauch	Samantha	Nathan@yesenia.net	{street: 'Douglas Extension', 'suite': 'Suite...', ...	1-463-123-4447	ramiro.info	{name: 'Romaguera-Jacobson', 'catchPhrase': '...
3	4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org	{street: 'Hoeger Mall', 'suite': 'Apt. 692', ...	493-170-9623 x156	kale.biz	{name: 'Robel-Corkery', 'catchPhrase': 'Mult...
4	5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	{street: 'Skiles Walks', 'suite': 'Suite 351', ...	(254)954-1289	demarco.info	{name: 'Keebler LLC', 'catchPhrase': 'User-c...

Merged all Sources:

```
47]:
```

```
# Merge ALL Sources
# Now combine data.

merged_df = df.merge(
    json_df,
    on="customer_id",
    how="left"
)

merged_df.head()
```

```
47]:
```

	customer_id	age	gender_x	region_x	education_level	employment_type	annual_income	loan_amount	loan_purpose	credit_score	repayment_history
0	1	59	Female	East	High School	Unemployed	213261	526963	Business	815	8
1	2	49	Male	West	Post-Graduate	Salaried	1325285	765732	Business	696	0
2	3	35	Male	East	Graduate	Salaried	568760	861527	Education	405	4
3	4	28	Female	West	Graduate	Salaried	1053605	361385	Car	384	2
4	5	41	Male	South	High School	Salaried	475796	677407	Car	513	5

```
49]: merged_df.shape
```

```
49]: (1000, 17)
```

Info:

```
merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   customer_id         1000 non-null   int64
1   age                 1000 non-null   int32
2   gender_x            1000 non-null   object
3   region_x            1000 non-null   object
4   education_level      1000 non-null   object
5   employment_type      1000 non-null   object
6   annual_income        1000 non-null   int32
7   loan_amount          1000 non-null   int32
8   loan_purpose          1000 non-null   object
9   credit_score         1000 non-null   int32
10  repayment_history    1000 non-null   int32
11  transaction_count    1000 non-null   int32
12  spending_ratio       1000 non-null   float64
13  join_date            1000 non-null   datetime64[ns]
14  default_flag         1000 non-null   int64
15  gender_y             5 non-null      object
16  region_y             5 non-null      object
dtypes: datetime64[ns](1), float64(1), int32(6), int64(2), object(7)
memory usage: 109.5+ KB
```